Amazon Engineering Challenge

Árvores Balanceadas

Duas estratégias de balanceamento.
Uma comparação com os mesmos dados.

AVL e Rubro-Negra

Davi Summer Silva Costa / 20250033590

Bacharelado em Tecnologia da Informação
Estruturas de Dados Básicas II · Prof.ª Ligia Francielle Borges

A ordem de chegada importa

Black Friday

Registrar, localizar e remover encomendas com eficiência.

Uma árvore de busca comum pode se alongar quando recebe chaves ordenadas.

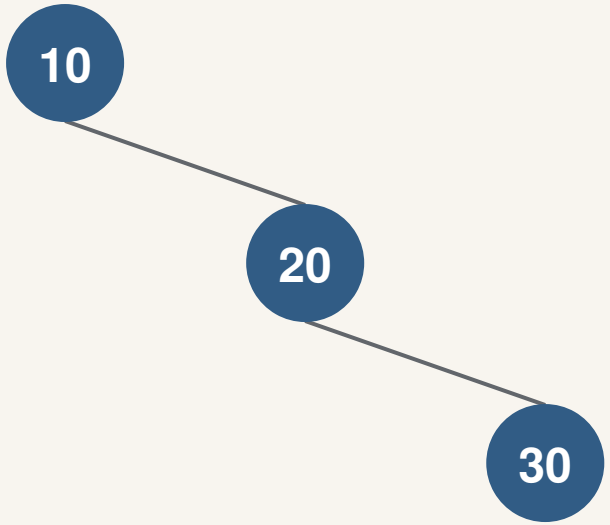
O protótipo compara AVL e rubro-negra para verificar como cada estrutura responde à mesma carga.

Chaves inteiras em memória. Sem persistência ou integração logística.

Fonte: enunciado da atividade, IMD0039, Unidade II

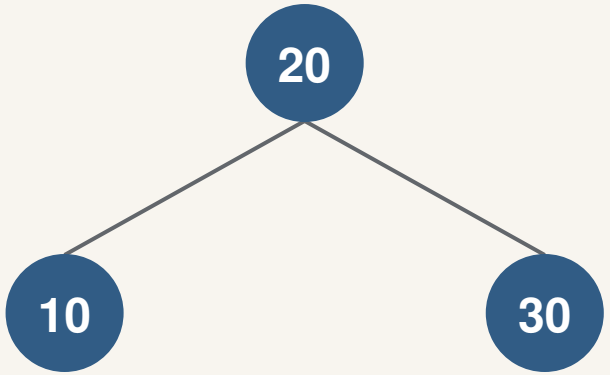
Por que balancear uma árvore de busca?

BST comum: inserir 10, 20, 30



Altura pode chegar a n nós.
Busca, inserção e remoção: O(n) no pior caso.

Árvore balanceada



Altura O(log n).
As três operações mantêm custo O(log n).

Base teórica: MIT 6.006, Lecture 7. Diagramas ilustrativos.

Uma interface, duas implementações

01	**avl.hpp**	Nós, rotações e balanceamento pela altura.
02	**rubro_negra.hpp**	Nós com cor, sentinela e ajustes rubro-negros.
03	**main.cpp**	Menu, medição e comparação das estruturas.

Inserir · buscar · remover · listar · imprimir · verificar

Duplicatas são ignoradas. Remoção ausente preserva o conjunto.
A impressão identifica as cores [PRETO] e [VERMELHO].

AVL: equilíbrio pela altura

FB = altura(esquerda) – altura(direita)

Cada nó deve manter FB entre –1 e +1. Após alterações, a implementação atualiza alturas e corrige os ancestrais.

Uma rotação

Esquerda–esquerda
Direita–direita

Duas rotações

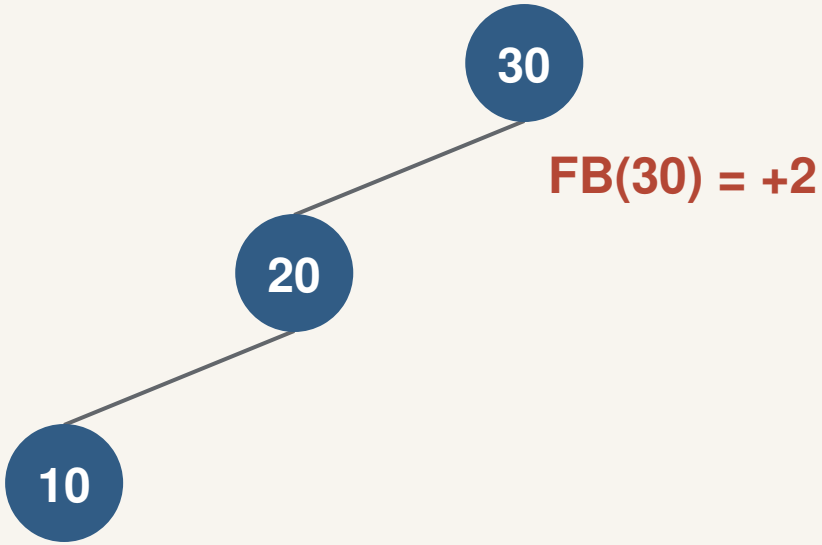
Esquerda–direita
Direita–esquerda

A rotação preserva o percurso em ordem. A remoção pode exigir ajustes em vários ancestrais.

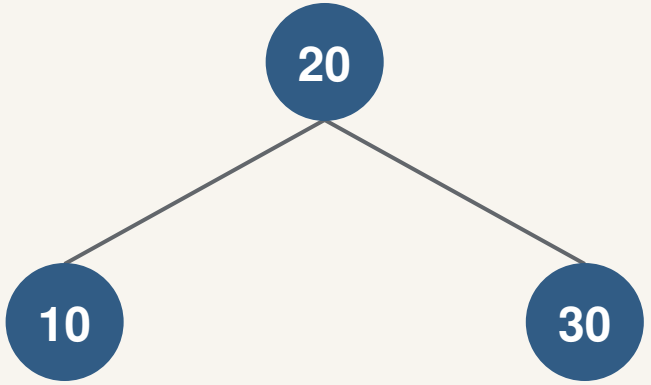
Fontes: src/avl.hpp; MIT 6.006, Lecture 7

AVL: rotação à direita em 30

Antes do ajuste



Depois do ajuste



FB = 0 nos três nós

Entrada: 30, 20, 10. Raiz final: 20. Uma rotação. Percurso: 10, 20, 30.

Rubro-negra: equilíbrio pelas cores

Cada nó é preto ou vermelho. A raiz e as folhas NIL são pretas.

Um nó vermelho não pode ter filho vermelho.

Os caminhos de um nó até suas folhas NIL têm a mesma quantidade de nós pretos.

Inserção e remoção restauram essas regras com recolorações e rotações.

O projeto usa uma sentinela preta para representar os filhos ausentes.

Fontes: src/rubro_negra.hpp; MIT 6.046J, Lecture 10

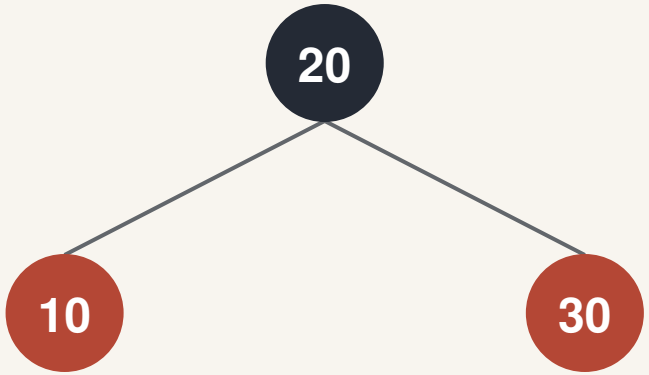
Rubro-negra: rotação e recoloração

Inserção de 10, antes do ajuste



Pai e filho vermelhos

Depois do ajuste



20 fica preto. 30 fica vermelho.
Rotação à direita em 30.

Entrada: 30, 20, 10. O tio NIL é preto. Folhas NIL omitidas no desenho.

Demonstração das duas árvores

Operação	AVL	Rubro-negra
Inserir 30, 20, 10, 5	Quatro inserções	Quatro inserções
Buscar 20 / buscar 99	Encontrado / ausente	Encontrado / ausente
Remover 20	Removido	Removido
Percurso final	5, 10, 30	5, 10, 30
Verificação final	valida=1	valida=1

Na rubro-negra, inserir 5 recolore 10 e 30 para preto.
A demonstração registra as cores e a estrutura após cada inserção.

Fonte: results/atual/demo.log. Síntese fiel da saída registrada.

O que os testes cobrem

Entradas e limites

Ordem crescente, decrescente e aleatória.

Árvore vazia, um nó e extremos inteiros.

Duplicatas e valores ausentes.

Mudanças na estrutura

Remoção de nós com 0, 1 e 2 filhos; remoção da raiz.

12.000 operações intercaladas por árvore, semente 2026.

Os quatro casos de rotação AVL.

std::set fornece o resultado de referência apenas nos testes.
A suíte confere o conjunto e as propriedades das árvores.

Validação concluída no Windows

27.202

verificações, zero falhas

Evidência	Resultado
operations.csv	27.100 operações com pass=1
comparison.csv	12 registros com valida=1
menu_test.log	PASS menu; execução com código 0

O total inclui verificações adicionais que não geram linhas no CSV de operações.

Experimento com entradas idênticas

1. Inserir 10.000 chaves distintas, de 1 a 10.000.

2. Buscar 10.000 existentes e 10.000 ausentes.

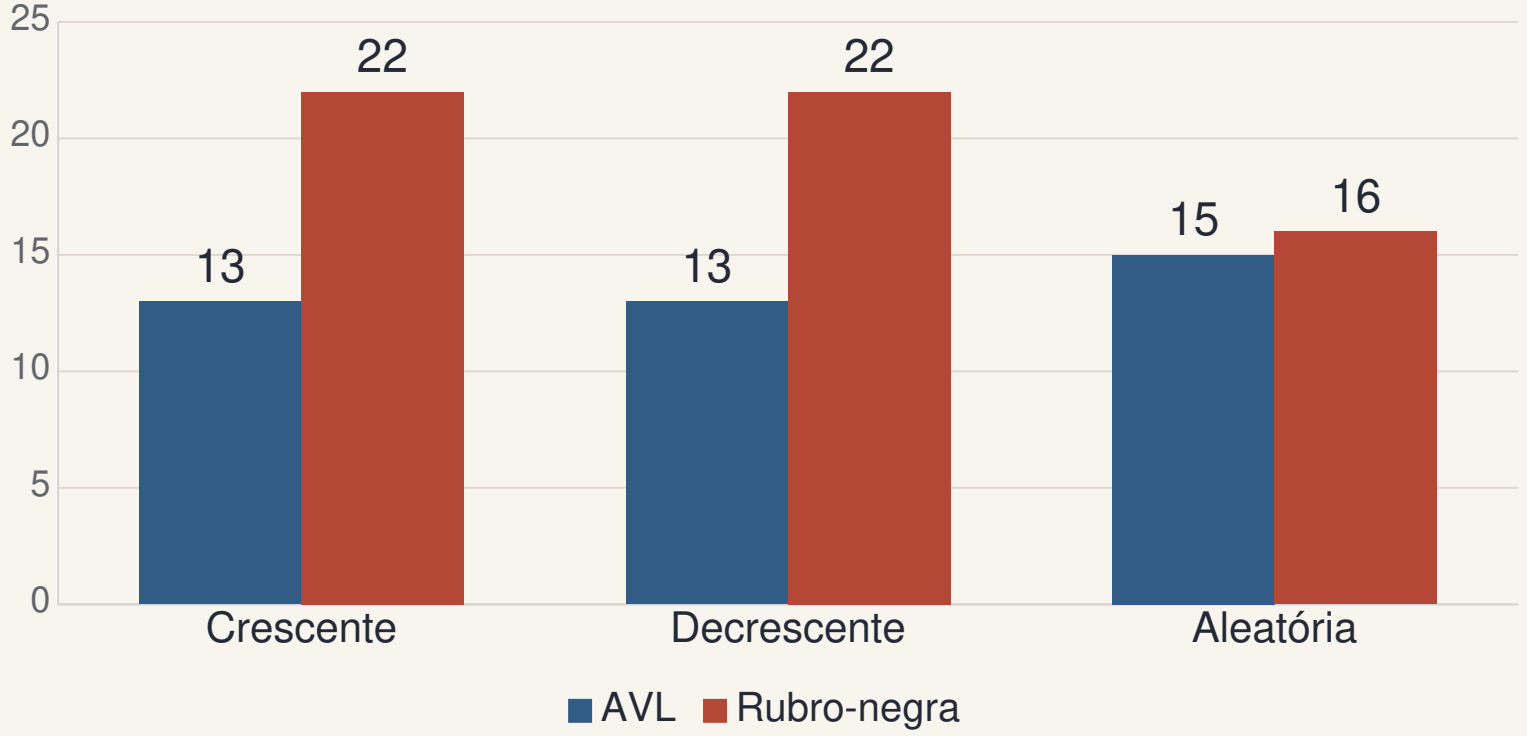
3. Remover as primeiras 5.000 chaves da sequência de entrada.

Repetir em ordem crescente, decrescente e aleatória.
Embaralhamento: mt19937, semente 42. Entradas salvas em TXT.

Altura em nós. Tempos totais dos lotes em ms. Contadores acumulados.
Uma execução no Windows, GCC 15.2.0 / UCRT64, C++17 e -O2.

Altura final: vantagem da AVL nas entradas ordenadas

Após remover metade: 5.000 nós



Após inserir 10.000 nós

Crescente e decrescente
AVL 14 / RN 24

Aleatória
AVL 16 / RN 16

Na entrada aleatória, ambas atingiram a mesma altura após a inserção.

Fonte: results/atual/comparison.csv. Alturas em nós.

Rotações ao longo do experimento

Entrada / árvore	Inserção	Só remoção	Total
crescente / AVL	9.986	2.497	12.483
crescente / Rubro-Negra	9.976	2.499	12.475
decrescente / AVL	9.986	2.497	12.483
decrescente / Rubro-Negra	9.976	2.499	12.475
aleatorio / AVL	6.968	2.153	9.121
aleatorio / Rubro-Negra	5.865	2.173	8.038

Inserção aleatória: AVL 6.968; rubro-negra 5.865 rotações.
A rubro-negra fez 1.103 rotações a menos nessa fase.

Fonte: comparison.csv. Rotações da remoção = total final – inserção.

O trabalho de recoloração

Entrada	Inserção	Só remoção	Total
crescente	49.877	14.996	64.873
decrescente	49.877	14.996	64.873
aleatorio	23.180	8.598	31.778

A rubro-negra combina rotações com ajustes de cor.
Na AVL, o acompanhamento usa alturas e fatores de balanceamento.

Menos rotações não significa, isoladamente, menor custo total.

Fonte: comparison.csv. Contagens da árvore rubro-negra.

Tempos observados nesta execução

Entrada / árvore	Inserir (ms)	Buscar (ms)	Remover (ms)
crescente / AVL	1,6906	0,5620	0,6306
crescente / Rubro-Negra	1,8073	0,9650	0,7197
decrescente / AVL	1,0946	0,3815	0,3391
decrescente / Rubro-Negra	1,9887	0,5898	0,6588
aleatorio / AVL	2,9505	0,7082	1,1456
aleatorio / Rubro-Negra	2,5903	1,0682	0,8681

A AVL teve menor tempo de busca nos três cenários. Na entrada aleatória, a rubro-negra teve menor tempo de inserção e remoção.

Fonte: comparison.csv. Lotes: 10.000 inserções, 20.000 buscas, 5.000 remoções.

Recomendação: AVL para priorizar consultas

Recomendo AVL se localizar encomendas for a prioridade.

Evidências: menor altura nos casos ordenados e menor tempo de busca nos três cenários medidos.

A rubro-negra merece avaliação para cargas com muitas atualizações: fez menos rotações na inserção aleatória e teve menor tempo de inserção e remoção nesse caso.

Limite: uma execução, sem variabilidade estatística nem carga real de produção. Antes de decidir em produção, repetir os ensaios e variar a proporção de operações.

Conclusão para o protótipo e as entradas testadas.

Como reproduzir os resultados

No Git Bash, na raiz do projeto:

```
export PATH="/c/msys64/ucrt64/bin:$PATH"
bash testar.sh
python tests/menu_test.py
```

O ajuste do PATH evita misturar a biblioteca C++ do Git com a do MSYS2 UCRT64. Os testes passaram com -O2 após esse ajuste.

Evidências: results/atual/
Executável: bin/encomendas.exe. A execução depende do ambiente descrito.

Referências e acesso ao projeto

Enunciado IMD0039: Amazon Engineering Challenge, Unidade II.
Código e evidências: src/, tests/ e results/atual/.

MIT 6.006 (2020): Lecture 7, Binary Trees II: AVL

MIT 6.046J (2005): Lecture 10, Red-black Trees

Os títulos acima contêm links para os materiais consultados.

Código, executável e resultados

https://github.com/davisummer156/amazon-engineering-challenge-arvores-balanceadas

Consulta às fontes teóricas: 28/10/2026